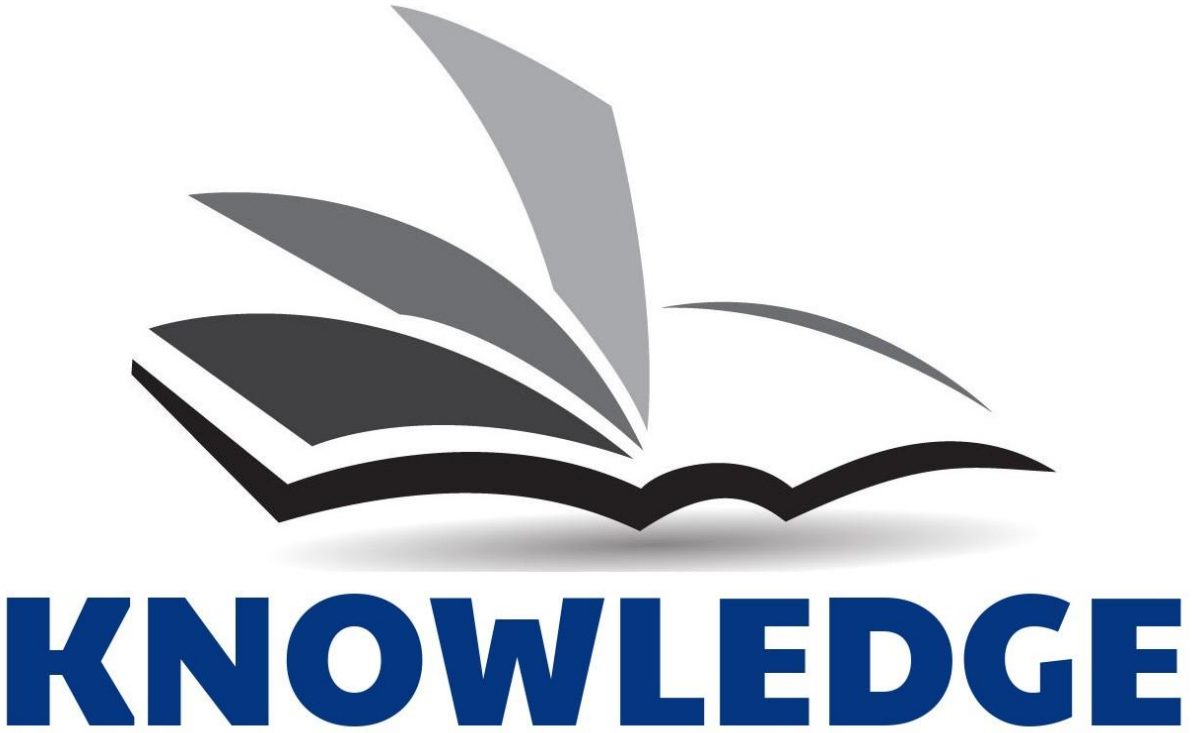
Dossier de Projet - Knowledge Learning



Plateforme E-Learning & E-Commerce

Table des matières

1. Présentation du projet 3

2. Résumé 3

3. Structure du repository 4

4. Le projet back-end 5

A. Modélisation et gestion de la base de données 5

A.2 Description détaillée des tables 6

A.3 Récapitulatif des contraintes d'intégrité 13

A.4 Paramètres techniques de la base 14

A.5 Sécurité de la base de données 14

A.6 Procédure de déploiement 19

A.7 Conformité et bonnes pratiques 20

A.8 Conclusion 20

B. API Platform 21

C. Architecture du code 21

C.1 Pattern utilisés 21

C.2 Composants E-commerce 26

C.3 Tests unitaires et fonctionnels 29

C.4 Sécurité 32

5. Le projet front-end 33

5.1 Conception UX/UI 33

5.2 Schéma de navigation 40

5.3 Développement et intégration 41

5.4 Architecture des templates 42

5.5 Gestion de la navigation et des données 44

5.6 Pages d'administration 45

6. Synthèse des compétences professionnelles 47

7. Annexes 48

1. PRÉSENTATION DU PROJET

🎓 Knowledge Learning

Plateforme E-Learning avec Paiement Intégré

Projet académique - Développement Backend Orienté Objet
Cadre : Projet d'évaluation - Conception d'une application e-commerce/e-learning

Contexte

L'entreprise fictive **Knowledge**, éditeur de livres de formation, souhaite étendre son offre en ligne via une plateforme e-learning permettant à ses clients d'étudier en autonomie depuis chez eux.

Mission

Concevoir et développer le **backend et frontend complet** de la plateforme :

- Base de données relationnelle optimisée
- Composants d'accès aux données (Repositories)
- Système de paiement intégré (Stripe)
- Architecture orientée objet maintenable

2. RÉSUMÉ DU PROJET

Knowledge Learning est une plateforme e-learning développée avec **Symfony 7.4** permettant aux utilisateurs d'acheter et suivre des formations en ligne. L'application propose une architecture hiérarchique Thème → Cursus → Leçons, avec un système d'authentification sécurisé nécessitant une activation par email. Les utilisateurs peuvent acheter des cursus complets ou des leçons individuelles via **Stripe** en mode sandbox. Un système de validation progressive des leçons permet l'obtention automatique de certifications « Knowledge Learning » après complétion d'un cursus. L'interface d'administration, développée avec **EasyAdminBundle**, offre une gestion complète des utilisateurs, contenus et transactions. Le projet respecte les principes SOLID avec une architecture 3-tiers (MVC + Service Layer), intègre des tests unitaires et fonctionnels (PHPUnit), et utilise des design patterns modernes (Action Controller, Repository, DTO). La sécurité est assurée par un hashage bcrypt des mots de passe, une protection CSRF native Symfony, et une gestion fine des autorisations avec les rôles ROLE_USER et ROLE_ADMIN.

3. Structure du repository

```
knowledge_learning/
├── src/
│   ├── Controller/      # Action Controllers (MVC)
│   ├── Entity/          # Modèles Doctrine ORM
│   ├── Repository/      # Composants d'accès aux données
│   ├── Service/         # Logique métier (Service Layer)
│   ├── DTO/             # Data Transfer Objects
│   ├── Security/        # Authenticator personnalisé
│   └── DataFixtures/    # Données de test
├── tests/
│   ├── Unit/            # Tests unitaires
│   └── Functional/      # Tests fonctionnels
├── templates/           # Vues Twig
├── migrations/          # Migrations Doctrine
├── config/              # Configuration Symfony
├── README.md            # Documentation d'installation
├── CONFIG_TEST.md       # Guide de configuration des tests
└── TESTS_ANALYSIS.md    # Analyse de la couverture de tests
```

Symfony impose une séparation claire des responsabilités. Le dossier `src/` contient les `Controller/` qui orchestrent les requêtes, les `Entity/` qui modélisent les données, les `Repository/` qui encapsulent les requêtes DB, et les `Service/` qui centralisent la logique métier. Les `DTO/` structurent les données entre couches, les `templates/` Twig génèrent la vue.

Cette architecture force une séparation nette : chaque couche a un rôle précis. Un contrôleur route la requête, un service exécute la logique, un repository accède aux données. Le code reste prévisible et navigable.

Les services sont réutilisables partout dans l'app, éliminant la duplication. Un DTO peut valider aussi bien une API qu'un formulaire. L'injection de dépendances du framework rend tout découplé et testable facilement.

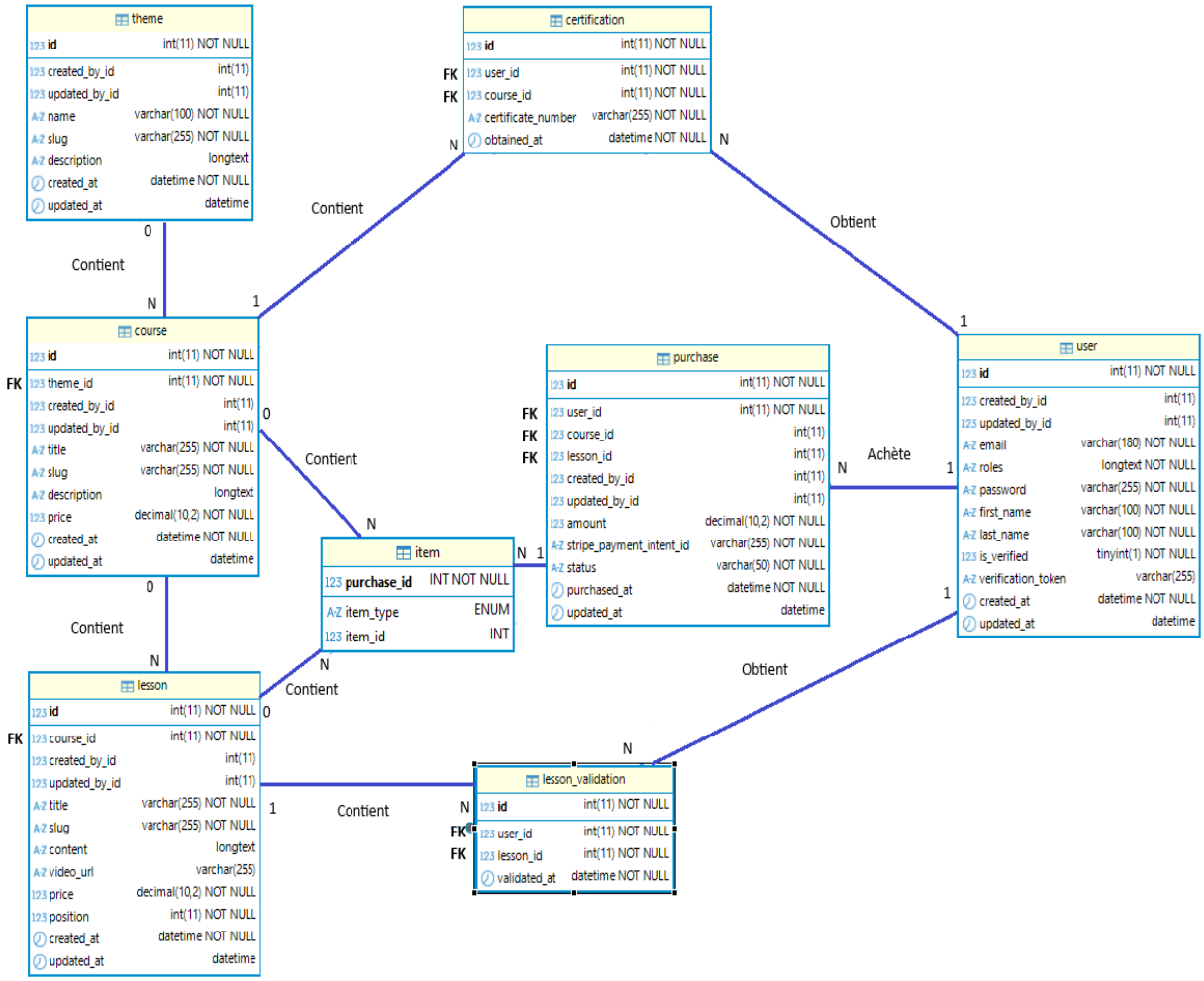
Quand le projet scale, j’ajoute simplement de nouveaux services ou contrôleurs sans refactoriser.

Au final, cette organisation c'est de la maintenabilité, de la scalabilité et de la testabilité inscrite dans le code dès le départ.

4. LE PROJET BACK-END

Architecture de l'Application

A. Modélisation et gestion de la base de données



L'accès aux données est assuré par l'ORM Doctrine, qui permet de manipuler les entités de l'application et d'interagir avec la base de données sans écrire directement de requêtes SQL. Doctrine facilite la gestion des relations entre les différentes entités et simplifie les opérations de lecture, création, modification et suppression des données.

La base de données est structurée de façon hiérarchique : un Theme regroupe un ou plusieurs Course (cursus), et chaque Course contient un ou plusieurs Lesson (leçons). Un User peut acheter un cursus et/ou une leçon via une Purchase et Item, puis valider sa progression au fil des LessonValidation. Une fois les leçons d'un cursus complétées, l'utilisateur obtient une Certification associée au cursus.

4.2 Description détaillée des tables

Table USER

Colonne	Type	Taille	Contraintes	Valeur par défaut	Description
id	INT	-	PRIMARY KEY, AUTO_INCREMENT, NOT NULL	-	Identifiant unique de l'utilisateur
email	VARCHAR	180	NOT NULL, UNIQUE (UNIQ_IDENTIFIER_EMAIL)	-	Adresse email (identifiant de connexion)
roles	JSON	-	NOT NULL	-	Rôles de l'utilisateur (ROLE_USER, ROLE_ADMIN, etc.)
password	VARCHAR	255	NOT NULL	-	Hash du mot de passe (bcrypt/argon2)
first_name	VARCHAR	100	NOT NULL	-	Prénom de l'utilisateur
last_name	VARCHAR	100	NOT NULL	-	Nom de famille de l'utilisateur
is_verified	TINYINT	1	NOT NULL	-	Statut de vérification du compte (0=non, 1=oui)
verification_token	VARCHAR	255	NULL	NULL	Token de vérification email (temporaire)
created_at	DATETIME	-	NOT NULL, IMMUTABLE	-	Date de création du compte

Index : UNIQ_IDENTIFIER_EMAIL (UNIQUE) sur email

Contraintes métier :

- L'email est unique et sert d'identifiant de connexion
- Le champ roles stocke un tableau JSON des rôles
- Le token de vérification est supprimé après validation

Table THEME

Colonne	Type	Taille	Contraintes	Valeur par défaut	Description
id	INT	-	PRIMARY KEY, AUTO_INCREMENT, NOT NULL	-	Identifiant unique du thème
name	VARCHAR	100	NOT NULL	-	Nom du thème (ex: "Développement Web")
slug	VARCHAR	255	NOT NULL	-	Identifiant URL-friendly (SEO)
description	LONGTEXT	-	NULL	NULL	Description détaillée du thème

Table COURSE

Colonne	Type	Taille	Contraintes	Valeur par défaut	Description
id	INT	-	PRIMARY KEY, AUTO_INCREMENT, NOT NULL	-	Identifiant unique du cours
theme_id	INT	-	NOT NULL, FOREIGN KEY → theme(id)	-	Référence vers le thème parent
title	VARCHAR	255	NOT NULL	-	Titre du cours
slug	VARCHAR	255	NOT NULL	-	Identifiant URL-friendly
description	LONGTEXT	-	NULL	NULL	Description détaillée du cours
price	NUMERIC	(10,2)	NOT NULL	-	Prix du cours entier (en euros)

Colonne	Type	Taille	Contraintes	Valeur par défaut	Description
created_at	DATETIME	-	NOT NULL, IMMUTABLE	-	Date de création du cours

Index : IDX_169E6FB959027487 sur theme_id

Clés étrangères : FK_169E6FB959027487 : theme_id → theme(id)

Table LESSON

Colonne	Type	Taille	Contraintes	Valeur par défaut	Description
id	INT	-	PRIMARY KEY, AUTO_INCREMENT, NOT NULL	-	Identifiant unique de la leçon
course_id	INT	-	NOT NULL, FOREIGN KEY → course(id)	-	Référence vers le cours parent
title	VARCHAR	255	NOT NULL	-	Titre de la leçon
slug	VARCHAR	255	NOT NULL	-	Identifiant URL-friendly
content	LONGTEXT	-	NULL	NULL	Contenu textuel de la leçon
video_url	VARCHAR	255	NULL	NULL	URL de la vidéo associée
price	NUMERIC	(10,2)	NOT NULL	-	Prix de la leçon à l'unité (en euros)
position	INT	-	NOT NULL	-	Ordre d'affichage dans le cours
created_at	DATETIME	-	NOT NULL, IMMUTABLE	-	Date de création de la leçon

Index : IDX_F87474F3591CC992 sur course_id

Clés étrangères : FK_F87474F3591CC992 : course_id → course(id)

Table PURCHASE

Colonne	Type	Taille	Contraintes	Valeur par défaut	Description
id	INT	-	PRIMARY KEY, AUTO_INCREMENT, NOT NULL	-	Identifiant unique de l'achat
user_id	INT	-	NOT NULL, FOREIGN KEY → user(id)	-	Utilisateur ayant effectué l'achat
amount	NUMERIC	(10,2)	NOT NULL	-	Montant payé (en euros)
stripe_payment_intent_id	VARCHAR	255	NOT NULL	-	Identifiant de paiement Stripe
status	VARCHAR	50	NOT NULL	-	Statut du paiement (pending, succeeded, failed)
purchased_at	DATETIME	-	NOT NULL, IMMUTABLE	-	Date et heure de l'achat
created_by	INT	-	FOREIGN KEY → user(id)	NULL	Audit
updated_by	INT		FOREIGN KEY → user(id)	NULL	Audit

Index :

- IDX_6117D13BA76ED395 sur user_id

Clés étrangères :

- FK_6117D13BA76ED395 : **user_id** → user(id)
- FK_6987D13DG76ED356 : **created_by** → user(id)
- FK_5447D13VC76ED785 : **updated_by** → user(id)

Table CERTIFICATION

Colonne	Type	Taille	Contraintes	Valeur par défaut	Description
id	INT	-	PRIMARY KEY, AUTO_INCREMENT, NOT NULL	-	Identifiant unique de la certification
user_id	INT	-	NOT NULL, FOREIGN KEY → user(id)	-	Utilisateur ayant obtenu la certification
course_id	INT	-	NOT NULL, FOREIGN KEY → course(id)	-	Cours certifié
certificate_number	VARCHAR	255	NOT NULL, UNIQUE	-	Numéro unique du certificat
obtained_at	DATETIME	-	NOT NULL, IMMUTABLE	-	Date d'obtention de la certification

Index :

- UNIQ_6C3C6D753005EFE3 (UNIQUE) sur certificate_number
- IDX_6C3C6D75A76ED395 sur user_id
- IDX_6C3C6D75591CC992 sur course_id
- user_course_unique (UNIQUE) sur (user_id, course_id)

Clés étrangères :

- FK_6C3C6D75A76ED395 : user_id → user(id)
- FK_6C3C6D75591CC992 : course_id → course(id)

Table LESSON_VALIDATION

Colonne	Type	Taille	Contraintes	Valeur par défaut	Description
id	INT	-	PRIMARY KEY, AUTO_INCREMENT, NOT NULL	-	Identifiant unique de la validation
user_id	INT	-	NOT NULL, FOREIGN KEY → user(id)	-	Utilisateur ayant validé la leçon
lesson_id	INT	-	NOT NULL, FOREIGN KEY → lesson(id)	-	Leçon validée
validated_at	DATE TIME	-	NOT NULL, IMMUTABLE	-	Date de validation de la leçon

Index :

- IDX_1BA512BFA76ED395 sur user_id
- IDX_1BA512BFCDF80196 sur lesson_id
- user_lesson_unique (UNIQUE) sur (user_id, lesson_id)

Clés étrangères :

- FK_1BA512BFA76ED395 : user_id → user(id)
- FK_1BA512BFCDF80196 : lesson_id → lesson(id)

Table ITEM

Colonne	Type	Taille	Contraintes	Valeur par défaut	Description
id	INT	-	PRIMARY KEY, AUTO_INCREMENT, NOT NULL	-	Identifiant unique
purchase_id	INT	-	FOREIGN KEY → purchase(id)	-NULL	Numéro de commande
lesson_id	INT	-	FOREIGN KEY → lesson(id)	-NULL	Leçon achetée
course_id	INT	-	FOREIGN KEY → course(id)	-NULL	Cursus acheté
unit_price	DECIMAL	10,2	NOT NULL	-	Prix unitaire
quantity	INT	-	NOT NULL	-	Quantité

- FK_1BA512BFA76ED395 : course_id → course(id)
- FK_1BA512BFCDF80196 : lesson_id → lesson(id)
- FK_1BA512BFCDF80258 : purchase_id → purchase(id)

4.3 Récapitulatif des contraintes d'intégrité

Contraintes d'unicité (UNIQUE)

Table	Contrainte	Colonnes	Justification
user	UNIQ_IDENTIFIER_EMAIL	email	Un email = un compte unique
certification	UNIQ_6C3C6D753005EFE3	certificate_number	Numéro de certificat unique
certification	user_course_unique	user_id, course_id	Une certification par cours et par utilisateur
lesson_validation	user_lesson_unique	user_id, lesson_id	Une validation par leçon et par utilisateur

Contraintes référentielles (FOREIGN KEY)

Table enfant	Colonne	Table parent
course	theme_id	theme(id)
lesson	course_id	course(id)
purchase	user_id	user(id)
item	purchase_id	purchase(id)
item	course_id	course(id)
item	lesson_id	lesson(id)
certification	user_id	user(id)
certification	course_id	course(id)
lesson_validation	user_id	user(id)
lesson_validation	lesson_id	lesson(id)

4.4 Paramètres techniques de la base

Moteur de stockage : InnoDB

- Support des transactions ACID (**atomicité, cohérence, isolement et durabilité)**
- Support des clés étrangères

Encodage : utf8mb4

- Support complet des caractères Unicode (emojis, caractères spéciaux internationaux)

Collation : utf8mb4_unicode_ci

- Comparaison insensible à la casse (Case Insensitive)
- Tri selon les règles Unicode standard

4.5 Sécurité de la base de données

A. Contexte et problématique

A.1 Problème identifié

L'utilisation du compte root MySQL dans une application présente des risques de sécurité majeurs :

- ✗ Accès non restreint à toutes les bases de données du serveur
- ✗ Capacité de modifier la configuration du serveur MySQL
- ✗ Possibilité de supprimer des bases de données d'autres applications
- ✗ Non-respect du principe du moindre privilège (*Least Privilege Principle*)

A.2 Solution mise en œuvre

Création d'un utilisateur MySQL dédié kl_app_user avec :

- ✓ Privilèges limités aux bases de données du projet uniquement
- ✓ Aucun accès aux autres bases de données du serveur
- ✓ Permissions strictement nécessaires au fonctionnement de l'application
- ✓ Respect des bonnes pratiques de sécurité

B. Configuration de la sécurité de la base

B.1 Script de création d'utilisateur

Le fichier database/setup_user.sql contient la configuration complète :

```
-- Création de l'utilisateur avec mot de passe sécurisé
CREATE USER IF NOT EXISTS 'kl_app_user'@'localhost'
IDENTIFIED BY 'Kn0wl3dg3_S3cur3!';

-- Privilège CREATE global (nécessaire pour Doctrine)
GRANT CREATE ON *.* TO 'kl_app_user'@'localhost';

-- Privilèges sur la base principale
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, INDEX, ALTER, DROP, REFERENCES
ON knowledge_learning.*
TO 'kl_app_user'@'localhost';

-- Privilèges sur la base de test
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, INDEX, ALTER, DROP, REFERENCES
ON knowledge_learning_test.*
TO 'kl_app_user'@'localhost';

-- Support des tests parallèles (pattern matching)
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, INDEX, ALTER, DROP, REFERENCES
ON `knowledge_learning_test%`.*
TO 'kl_app_user'@'localhost';

-- Application des privilèges
FLUSH PRIVILEGES;
```

B.2 Privilèges accordés

Privilège	Portée	Justification
CREATE	Global (*.*)	Permet à Doctrine de créer des bases de test (doctrine:database:create)
SELECT, INSERT, UPDATE, DELETE	knowledge_learning.*	Opérations CRUD sur les données
CREATE, ALTER, DROP	knowledge_learning.*	Gestion du schéma (migrations Doctrine)
INDEX	knowledge_learning.*	Optimisation des performances (index)
REFERENCES	knowledge_learning.*	Contraintes de clés étrangères

B.3 Configuration Symfony

Fichier `.env.local` (version sécurisée) :

```
DATABASE_URL="mysql://kl_app_user:Kn0wl3dg3_S3cur3!@127.0.0.1:3306/knowledge_learning?serverVersion=mariadb-10.4.32&charset=utf8mb4"
```

C. Restriction d'accès

C.1 Vérification des privilèges

Commande SQL :

```
SHOW GRANTS FOR 'kl_app_user'@'localhost';
```

Résultat :

```
+-----------------------------------------------------------------------+
| Grants for kl_app_user@localhost                                      |
+-----------------------------------------------------------------------+
| GRANT CREATE ON *.* TO `kl_app_user`@`localhost`                      |
| GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, REFERENCES,       |
|      INDEX, ALTER ON `knowledge_learning`.* TO `kl_app_user`@`localhost` |
| GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, REFERENCES,       |
|      INDEX, ALTER ON `knowledge_learning_test`.* TO `kl_app_user`@`localhost` |
+-----------------------------------------------------------------------+
```

C.2 Test d'isolation - Accès à knowledge_learning (autorisé)

Commande :

```
mysql -u kl_app_user -p knowledge_learning
```

Résultat SQL :

```
mysql> USE knowledge_learning;
Database changed

mysql> SELECT COUNT(*) FROM user;
+----------+
| COUNT(*) |
```

```
+----------+
|       5 |
+----------+
1 row in set (0.00 sec)
```

✓ **Résultat : SUCCÈS** - L'utilisateur peut accéder aux données de knowledge_learning

C.3 Test d'isolation - Accès à stubborn_shop (refusé)

Commande :

```
mysql> USE stubborn_shop;
Database changed

mysql> SHOW TABLES;
+---------------------------+
| Tables_in_stubborn_shop   |
+---------------------------+
| product                   |
| order                     |
| user                      |
+---------------------------+

mysql> SELECT * FROM product;
ERROR 1142 (42000): SELECT command denied to user 'kl_app_user'@'localhost'
for table `stubborn_shop`.`product`
```

✓ **Résultat : ACCÈS REFUSÉ** - L'utilisateur ne peut PAS lire les données d'autres bases

C.4 Test d'isolation - Tentative de suppression d'une autre base

Commande :

```
mysql> DROP DATABASE stubborn_shop;
ERROR 1044 (42000): Access denied for user 'kl_app_user'@'localhost'
to database 'stubborn_shop'
```

✓ **Résultat : ACCÈS REFUSÉ** - L'utilisateur ne peut PAS supprimer d'autres bases

C.5 Test de connexion Doctrine

Commande :

```
php bin/console dbal:run-sql "SELECT COUNT(*) as total FROM user"
```

Résultat :

```
-------
 total
-------
 5
-------
```

✓ Symfony se connecte correctement avec `kl_app_user`

C.6 Récapitulatif du moindre privilège

Capacité	root	kl_app_user
Créer des bases de données	✓	✓ (limité)
Lire/écrire dans knowledge_learning	✓	✓
Lire/écrire dans d'autres bases	✓	✗
Modifier les utilisateurs MySQL	✓	✗
Arrêter le serveur MySQL	✓	✗
Accéder aux tables système (mysql.*)	✓	✗

4.6 Procédure de déploiement

A Installation sur un nouvel environnement

Étapes à suivre :

```
# 1. Exécuter le script de création d'utilisateur
mysql -u root -p < database/setup_user.sql

# 2. Vérifier que l'utilisateur est créé
mysql -u root -p -e "SHOW GRANTS FOR 'kl_app_user'@'localhost';"

# 3. Tester la connexion
mysql -u kl_app_user -p knowledge_learning

# 4. Configurer Symfony
cp .env.local.example .env.local
# Éditer .env.local avec les bons identifiants

# 5. Créer la base de données
php bin/console doctrine:database:create

# 6. Exécuter les migrations
php bin/console doctrine:migrations:migrate

# 7. Charger les fixtures (optionnel)
php bin/console doctrine:fixtures:load
```

4.7. Conformité et bonnes pratiques

A. Standards de sécurité respectés

- ✓ **OWASP Top 10** : Protection contre l'accès non autorisé aux données
- ✓ **ISO 27001** : Contrôle d'accès basé sur les rôles
- ✓ **RGPD** : Limitation de l'accès aux données personnelles
- ✓ **CWE-250** : Exécution avec des privilèges inutiles

B. Recommandations supplémentaires

Pour renforcer encore la sécurité :

1. **En production :** Utiliser un mot de passe différent et plus complexe
2. **Monitoring :** Surveiller les tentatives de connexion échouées
3. **Rotation :** Changer le mot de passe régulièrement (tous les 90 jours)

4.8. Conclusion

La mise en place d'un utilisateur MySQL dédié avec des privilèges restreints permet de :

1. **Réduire la surface d'attaque** : Limitation des capacités en cas de compromission
2. **Isoler les données** : Protection des autres bases de données du serveur
3. **Traçabilité** : Identification claire de l'application accédant aux données
4. **Conformité** : Respect des normes de sécurité et des bonnes pratiques

---

B. API Platform

Cette mini-API a été construite avec API Platform pour exposer des données “publiables” sans donner accès directement aux entités Doctrine. La ressource exposée est un DTO (LessonOutputDto) alimenté par un Provider (LessonOutputProvider) qui récupère et transforme les entités Lesson.

L’API est montée sous le préfixe /api via la config routes API Platform :

```yaml
# config/routes/api_platform.yaml
api_platform:
    resource: .
    type: api_platform
    prefix: /api
```

Cela active automatiquement la doc OpenAPI/Swagger et la génération des endpoints déclarés avec #[ApiResource].

La ressource publique n’est pas une entité Doctrine, mais un DTO.

C’est ce DTO qui définit les routes exposées :

```php
<?php

namespace App\DTO\Api;

use ApiPlatform\Metadata\ApiProperty;
use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Get;
use ApiPlatform\Metadata\GetCollection;
use App\State\Api\LessonOutputProvider;

Mathieu P.
#[ApiResource(
    shortName: 'Lesson',
    operations: [
        new Get(uriTemplate: '/lessons/{id}', provider: LessonOutputProvider::class),
        new GetCollection(uriTemplate: '/lessons', provider: LessonOutputProvider::class),
    ],
    formats: ['json' => ['application/json']],
)]
final class LessonOutputDto
{
    Mathieu P.
    public function __construct(
        #[ApiProperty(identifier: true)]
        public int $id,
        public string $title,
        public string $price,
        public CourseSummaryDto $course,
    ) {}
}
```

Le provider remplace le mécanisme standard. Il applique pagination, tri, filtres, puis mappe chaque entité Lesson vers le DTO.

```php
public function provide(Operation $operation, array $uriVariables = [], array $context = []): LessonOutputDto|array|null
{
    $filters = $context['filters'] ?? [];

    if ($operation instanceof CollectionOperationInterface) {
        $page = max( value: 1, (int)($filters['page'] ?? 1));
        $itemsPerPage = min( value: 50, max( value: 1, (int)($filters['itemsPerPage'] ?? 10)));

        $sort = $filters['sort'] ?? 'position';
        $sort = in_array($sort, ['id', 'title', 'price', 'position'], strict: true) ? $sort : 'position';

        $direction = strtoupper((string)($filters['direction'] ?? 'ASC')) === 'DESC' ? 'DESC' : 'ASC';

        $courseId = isset($filters['course']) ? (int)$filters['course'] : null;
        $minPrice = isset($filters['minPrice']) ? (float)$filters['minPrice'] : null;
        $maxPrice = isset($filters['maxPrice']) ? (float)$filters['maxPrice'] : null;

        $lessons = $this->lessonRepository->searchApiLessons(
            page: $page,
            itemsPerPage: $itemsPerPage,
            sort: $sort,
            direction: $direction,
            courseId: $courseId,
            minPrice: $minPrice,
            maxPrice: $maxPrice
        );

        return array_map(fn (Lesson $lesson) => $this->map($lesson), $lessons);
    }
```

Le mapping final vers DTO est fait ici :

```php
private function map(Lesson $lesson): LessonOutputDto
{
    $course = $lesson->getCourse();

    return new LessonOutputDto(
        id: (int)$lesson->getId(),
        title: (string)$lesson->getTitle(),
        price: (string)$lesson->getPrice(),
        course: new CourseSummaryDto(
            id: (int)$course?->getId(),
            title: (string)$course?->getTitle(),
            description: (string)$course?->getDescription(),
            price: (string)$course?->getPrice(),
        )
    );
}
```

Le repository applique les filtres et le tri en SQL/Doctrine :

```php
public function searchApiLessons(
    int $page = 1,
    int $itemsPerPage = 10,
    string $sort = 'position',
    string $direction = 'ASC',
    ?int $courseId = null,
    ?float $minPrice = null,
    ?float $maxPrice = null
): array {
    $page = max( value: 1, $page);
    $itemsPerPage = min( value: 50, max( value: 1, $itemsPerPage));

    // Whitelist anti-injection sur le champ de tri
    $allowedSorts = [
        'id' => 'l.id',
        'title' => 'l.title',
        'price' => 'l.price',
        'position' => 'l.position',
    ];
    $sortField = $allowedSorts[$sort] ?? $allowedSorts['position'];

    $direction = strtoupper($direction) === 'DESC' ? 'DESC' : 'ASC';

    $qb = $this->createQueryBuilder( alias: 'l')
        ->leftJoin( join: 'l.course',  alias: 'c')
        ->addSelect( select: 'c');

    if ($courseId !== null) {
        $qb->andWhere('c.id = :courseId')
            ->setParameter( key: 'courseId', $courseId);
    }
    if ($minPrice !== null) {
        $qb->andWhere('l.price >= :minPrice')
            ->setParameter( key: 'minPrice', $minPrice);
    }

    if ($maxPrice !== null) {
        $qb->andWhere('l.price <= :maxPrice')
            ->setParameter( key: 'maxPrice', $maxPrice);
    }

    return $qb
        ->orderBy($sortField, $direction)
        ->setFirstResult( firstResult: ($page - 1) * $itemsPerPage)
        ->setMaxResults($itemsPerPage)
        ->getQuery()
        ->getResult();
}
```

Sécurité et CORS

L'API est stateless, le serveur ne garde pas d'état de session entre deux requêtes. :

```
# config/packages/api_platform.yaml
api_platform:
    defaults:
        stateless: true
```

Aucune règle access_control n'est définie pour /api, donc l'API est publique.

CORS (Cross-Origin Resource Sharing) est activé globalement via Nelmio :

```
# config/packages/nelmio_cors.yaml
nelmio_cors:
    defaults:
        allow_origin: ['%env(CORS_ALLOW_ORIGIN)%']
        allow_methods: ['GET', 'OPTIONS', 'POST', 'PUT', 'PATCH', 'DELETE']
        allow_headers: ['Content-Type', 'Authorization']
```

API Platform génère automatiquement la doc :

Swagger UI : /api/docs

OpenAPI JSON : /api/docs.json

La doc reflète directement les opérations définies sur LessonOutputDto et le schéma du DTO.

Conclusion

L'API est volontairement minimaliste et contrôlée : on expose un DTO et non les entités, avec un provider sur mesure qui gère le tri, les filtres et la pagination.

La doc est auto-générée via API Platform, et l'API est publique par défaut (pas d'auth).

C'est une base propre pour montrer la "construction d'une API" sans impacter le reste du projet.

Résultat d'une requête via Postman :

GET http://127.0.0.1:8000/api/lessons Send

{} JSON Preview Visualize


```
1  [
2      {
3          "id": 1,
4          "title": "Découverte de l'instrument",
5          "price": "26.00",
6          "course": {
7              "id": 1,
8              "title": "Cursus d'initiation à la guitare",
9              "description": "Un cursus complet pour maîtriser les fondamentaux.",
10             "price": "50.00"
11         }
12     },
13     {
14         "id": 3,
15         "title": "Découverte de l'instrument",
16         "price": "26.00",
17         "course": {
18             "id": 2,
19             "title": "Cursus d'initiation au piano",
20             "description": "Un cursus complet pour maîtriser les fondamentaux.",
21             "price": "50.00"
22         }
23     },
```

C. Architecture du Code

C.1 Patterns Utilisés

1. **Action Controller Pattern**
   - Un contrôleur = une action (__invoke())
   - Injection de dépendances via le constructeur
   - Exemple :

```php
#[Route('/', name: 'app_home')]
class HomeController extends AbstractController
{
    Mathieu P. *
    public function __invoke(CourseService $courseService): Response
    {
        $themes = $courseService->getAllThemes();

        return $this->render( view: 'home/index', [
            'themes' => $themes,
        ]);
    }
}
```

2. **Service Layer Pattern**
   - Services readonly (immutables)
   - Logique métier isolée des contrôleurs
   - Exemple :

```php
readonly class CourseService
{
    Mathieu P.
    public function __construct(
        private CourseRepository $courseRepository,
        private ThemeRepository  $themeRepository
    ) {
    }

    2 usages  Mathieu P.
    public function getAllThemes(): array
    {
        return $this->themeRepository->findAllThemes();
    }
```

3. **Repository Pattern (Doctrine ORM)**
   - Séparation de la couche d'accès aux données
   - Requêtes personnalisées optimisées

```
class CourseRepository extends ServiceEntityRepository
{
    Mathieu P.
    public function __construct(ManagerRegistry $registry)
    {
        parent::__construct($registry, Course::class);
    }

    2 usages   Mathieu P.
    public function findBySlug(string $slug): ?Course
    {
        return $this->createQueryBuilder( alias: 'c')
            ->leftJoin( join: 'c.theme',  alias: 't')
            ->addSelect( select: 't')
            ->leftJoin( join: 'c.lessons',  alias: 'l')
            ->addSelect( select: 'l')
            ->where( predicates: 'c.slug = :slug')
            ->setParameter( key: 'slug', $slug)
            ->getQuery()
            ->getOneOrNullResult();
    }
```

4. **DTO Pattern (Data Transfer Object)**
   - Validation des données entrantes
   - Séparation entités/formulaires

```
class RegistrationDTO
{
    2 usages
    #[Assert\NotBlank(message: 'Le prénom est obligatoire.')]
    #[Assert\Length(min: 2, max: 100)]
    private ?string $firstName = null;

    2 usages
    #[Assert\NotBlank(message: 'Le nom est obligatoire. ')]
    #[Assert\Length(min: 2, max: 100)]
    private ?string $lastName = null;

    2 usages
    #[Assert\NotBlank(message: 'L\'email est obligatoire.')]
    #[Assert\Email(message: 'L\'email n\'est pas valide.')]
    private ?string $email = null;

    2 usages
    #[Assert\NotBlank(message: 'Le mot de passe est obligatoire.')]
    #[Assert\Length(min: 8, minMessage: 'Le mot de passe doit contenir au moins 8 caractères.')]
    #[Assert\Regex(
        pattern: '/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)/',
        message: 'Le mot de passe doit contenir au moins une majuscule, une minuscule et un chiffre.'
    )]
    private ?string $password = null;
```

C.2. Composants E-Commerce

Intégration Stripe
Service de Paiement (StripeService.php)

```php
readonly class StripeService
{
    new *
    public function __construct(
        private ParameterBagInterface $params,
        private UrlGeneratorInterface $urlGenerator,
        private PurchaseRepository    $purchaseRepository
    ) {
        Stripe::setApiKey($this->params->get('stripe.secret_key'));
    }

    no usages  new *
    public function createCheckoutSession(
        User $user,
        ?Course $course = null,
        ?Lesson $lesson = null
    ): Session {
        // Création de la session Stripe Checkout
        return Session::create([
            'payment_method_types' => ['card'],
            'line_items' => [/* ... */],
            'mode' => 'payment',
            'success_url' => $successUrl,
            'cancel_url' => $cancelUrl,
            'metadata' => $metadata,
        ]);
    }
}
```

Gestion des Webhooks (WebhookHandlerService.php)

- Vérification de la signature Stripe
- Traitement événement checkout.session.completed
- Création automatique des Purchase en base

C.3. Tests Unitaires et Fonctionnels

Couverture des Tests

Composant	Type	Fichier	Statut
Inscription	Unitaire	RegistrationServiceTest.php	✅
Vérification email	Unitaire	VerificationServiceTest.php	✅
Connexion	Fonctionnel	LoginTest.php	✅
Achats	Unitaire + Fonctionnel	PurchaseServiceTest.php, PurchaseFlowTest.php	✅
Repositories	Unitaire	UserRepositoryTest.php, CourseRepositoryTest.php, PurchaseRepositoryTest.php	✅

6.1 Exemple de Test Unitaire

```php
class PurchaseServiceTest extends TestCase
{
    no usages  new *
    public function testCreatePurchaseForCourse(): void
    {
        // Arrange
        $user = $this->createUser();
        $course = $this->createCourse();

        // Act
        $purchase = $this->purchaseService->createPurchase(
            $user, 'pi_test_123', '50.00', $course, null
        );

        // Assert
        $this->assertInstanceOf(Purchase::class, $purchase);
        $this->assertEquals('50.00', $purchase->getAmount());
        $this->assertEquals('completed', $purchase->getStatus());
    }
}
```

```
$ php bin/phpunit
PHPUnit 9.6.31 by Sebastian Bergmann and contributors.

...............................                                   31 / 31 (100%)

Time: 00:08.589, Memory: 96.00 MB

OK (31 tests, 91 assertions)
```

6.2 Présentation du test fonctionnel LoginTest

Ce test valide le parcours de connexion/déconnexion côté utilisateur. Il vérifie l'accessibilité de la page de login, les cas de connexion correcte/incorrecte, et la déconnexion.

1) La page de connexion est accessible

Objectif : vérifier que la page /connexion s'affiche et contient les champs attendus.

```
$crawler = $client->request( method: 'GET',  uri: '/connexion');

$this->assertResponseIsSuccessful();
$this->assertSelectorTextContains( selector: 'h2',  text: 'Connexion');
$this->assertCount( expectedCount: 1, $crawler->filter( selector: 'input[name="login[email]"]'));
$this->assertCount( expectedCount: 1, $crawler->filter( selector: 'input[name="login[password]"]'));
```

2) Connexion avec identifiants valides

Objectif : s'assurer qu'un utilisateur vérifié peut se connecter.

```
$email = 'test.login' . uniqid() . '@example.com';
$user = $this->createVerifiedUser($email,  password: 'Password123!');

$crawler = $client->request( method: 'GET',  uri: '/connexion');

$form = $crawler->selectButton( value: 'Se connecter')->form([
    'login[email]' => $email,
    'login[password]' => 'Password123!',
]);

$client->submit($form);

$this->assertResponseRedirects();
$client->followRedirect();
$this->assertResponseIsSuccessful();
```

3) Connexion avec mot de passe invalide

Objectif : vérifier qu'un mauvais mot de passe est rejeté.

```
$form = $crawler->selectButton( value: 'Se connecter')->form([
    'login[email]' => $email,
    'login[password]' => 'WrongPassword',
]);

$client->submit($form);

$this->assertResponseRedirects( expectedLocation: '/connexion');
$client->followRedirect();
$this->assertSelectorExists( selector: '.alert-danger');
```

4) Connexion avec utilisateur inexistant

Objectif : vérifier le comportement quand l'email n'existe pas.

```
$form = $crawler->selectButton( value: 'Se connecter')->form([
    'login[email]' => 'nonexistent@example.com',
    'login[password]' => 'Password123!',
]);
```

5) Déconnexion

Objectif : s'assurer qu'un utilisateur connecté peut se déconnecter et perd l'accès aux pages protégées.

```
$client->loginUser($user);

$client->request( method: 'GET',  uri: '/mon-compte');
$this->assertResponseIsSuccessful();

$client->request( method: 'GET',  uri: '/logout');
$this->assertResponseRedirects();

$client->request( method: 'GET',  uri: '/mon-compte');
$this->assertResponseRedirects( expectedLocation: '/connexion');
```

Helper : création d'utilisateur vérifié

Le test utilise un helper pour créer un utilisateur validé rapidement :

```
$user = new User();
$user->setFirstName( firstName: 'Test');
$user->setLastName( lastName: 'User');
$user->setEmail($email);
$user->setPassword($passwordHasher->hashPassword($user, $password));
$user->setVerified( isVerified: true);

$entityManager->persist($user);
$entityManager->flush();

return $user;
```

C.4. Sécurité

Mesures Implémentées

1. **Protection CSRF**
   - Token CSRF automatique sur tous les formulaires (Symfony)
   - Validation côté serveur
2. **Hashage des Mots de Passe**
   - Algorithme **bcrypt** (Symfony PasswordHasher)
   - Salage automatique (Valeur aléatoire différente pour chaque mot de passe, hash différent même si il y a des mots de passe identique)
3. **Validation des Mots de Passe**
   - Minimum 8 caractères
   - Au moins 1 majuscule, 1 minuscule, 1 chiffre
   - Validation via Assert\Regex

```
#[Assert\Regex(
    pattern: '/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)/',
    message: 'Le mot de passe doit contenir majuscule, minuscule et chiffre.'
)]
public ?string $password = null;
```

4. **Gestion des Rôles**

   Pour assurer la sécurité de l'application et contrôler les différents niveaux d'accès, un système de rôles a été mis en place à l'aide du composant Symfony Security. Deux types de profils utilisateurs sont distingués dans l'application :

   - ROLE_USER : accès utilisateur standard
   - ROLE_ADMIN : accès backoffice EasyAdmin
   - Hiérarchie : ROLE_ADMIN hérite de ROLE_USER

```
# config/packages/security.yaml
access_control:
    - { path: ^/admin, roles: ROLE_ADMIN }
    - { path: ^/mon-compte, roles: ROLE_USER }
    - { path: ^/cursus/.*/acheter, roles: ROLE_USER }
```

5. **Activation par Email**
   - Token UUID v4 généré à l'inscription
   - Lien d'activation unique et temporaire
6. **CSP avec NelmioSecurityBundle**

```
nelmio_security:
    content_security_policy:
        defaults:
            - 'self'
```

5. LE PROJET FRONT-END

Le développement du **front-end** de la plateforme Knowledge Learning a constitué une étape essentielle afin de garantir une expérience utilisateur fluide, intuitive et adaptée à différents supports (desktop, tablette, mobile). Le choix technologique s'est porté sur **Twig**, le moteur de templates de Symfony, associé à **Bootstrap 5**, pour la conception d'interfaces responsives et modernes.

5.1 Conception UX/UI

La conception a été réalisée **directement en intégrant l'interface** en s'appuyant sur les composants Bootstrap 5 et une charte graphique cohérente. Voici les éléments de design :

Palette de couleurs :

Couleur	Hex	Utilisation
Bleu principal	#0d6efd	Boutons, liens, accents
Bleu foncé	#003d82	Header, navigation
Gris	#f8f9fa	Backgrounds
Vert succès	#28a745	Validations, badges
Rouge alerte	#dc3545	Erreurs, suppressions
Noir	#1a1a1a	Footer, textes

Typographie :

- **Font principale :** System font stack (Bootstrap default)
- **Logo :** Avenir (custom)

Identité visuelle :

- Logo avec livre stylisé
- Design épuré et professionnel
- Mise en avant des contenus pédagogiques

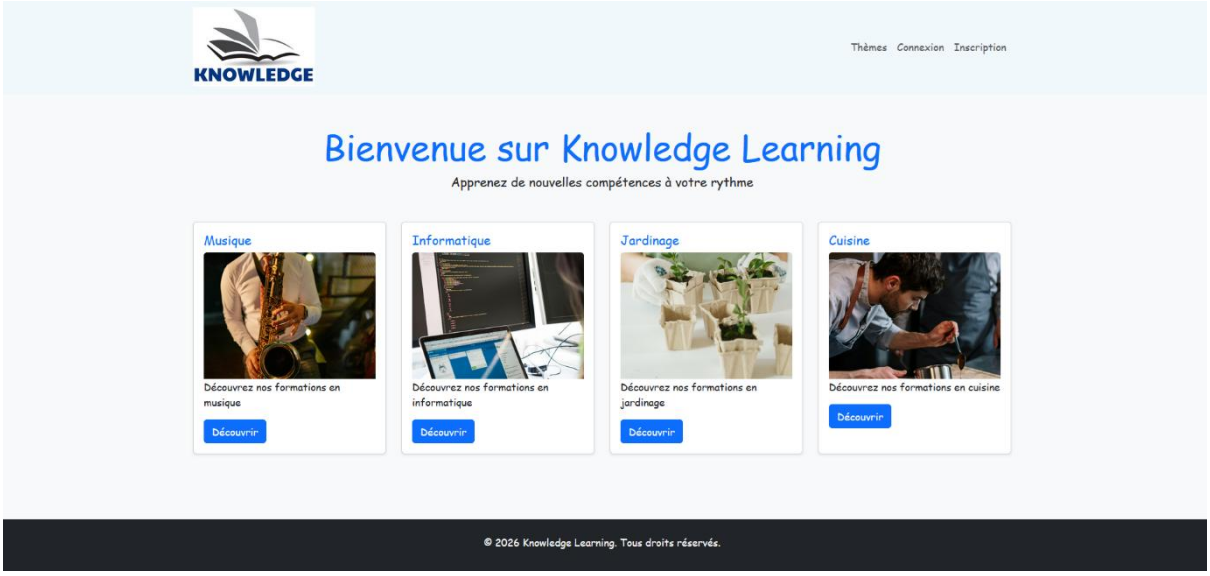
Les interfaces ont été pensées pour mettre en valeur la simplicité d'usage :

- une **page d'accueil** claire donnant accès aux cursus par thèmes,
- des **pages dédiées** à l'achat et à la consultation des cours,
- un **espace personnel** permettant de suivre la progression de l'apprenant,
- un **espace administrateur** distinct, pour la gestion des utilisateurs, des cursus et des commandes.

Les interfaces ont été déclinées en **version desktop** et en **version mobile**, afin d'assurer une adaptation optimale à toutes les résolutions d'écran. En voici quelques exemples :

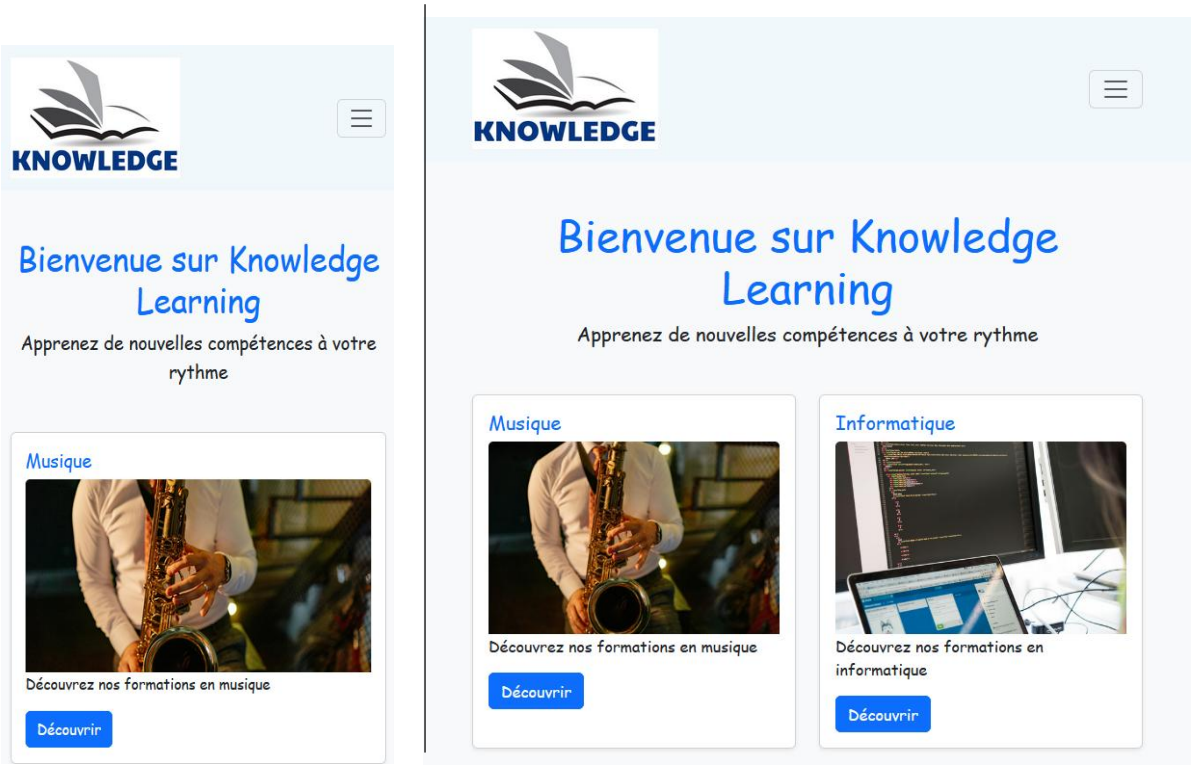
Captures d'écran des interfaces principales

Page d'accueil - Vue desktop

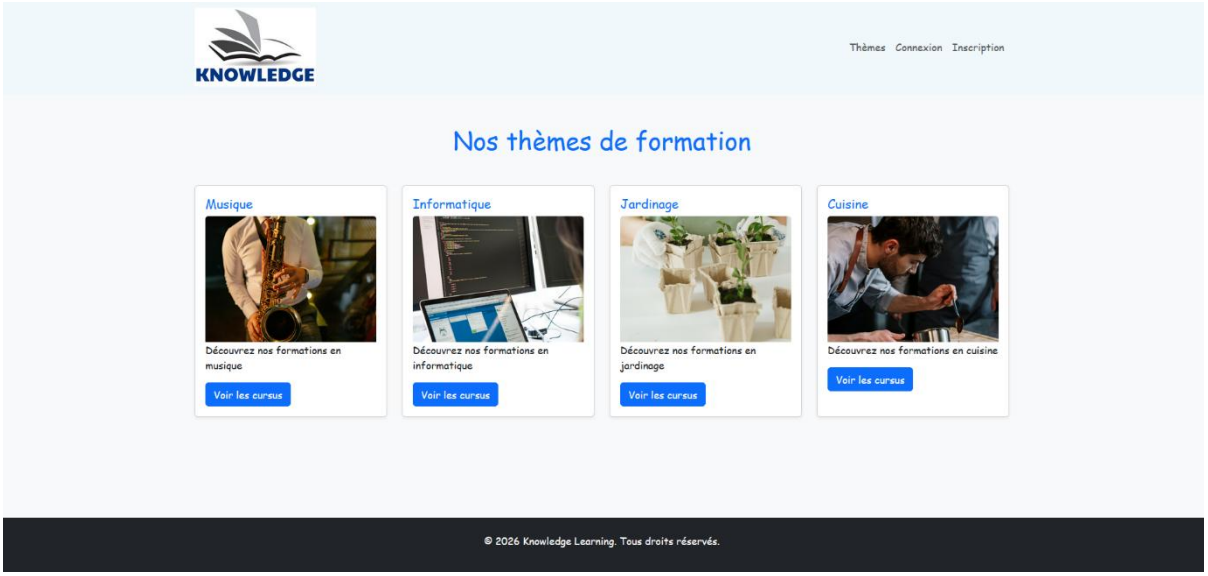


L'accueil présente les 4 thèmes de formation (Musique, Informatique, Jardinage, Cuisine) avec une grille responsive Bootstrap. Navigation conditionnelle selon l'authentification.

Vue mobile - tablette

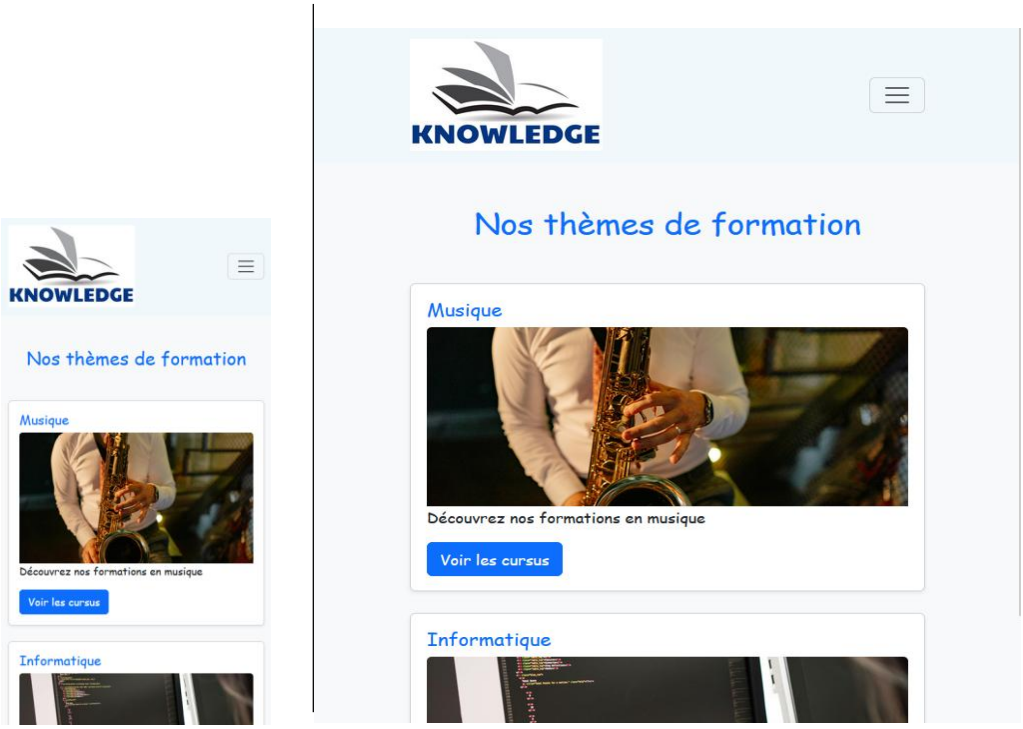


Page de liste des thèmes (desktop)

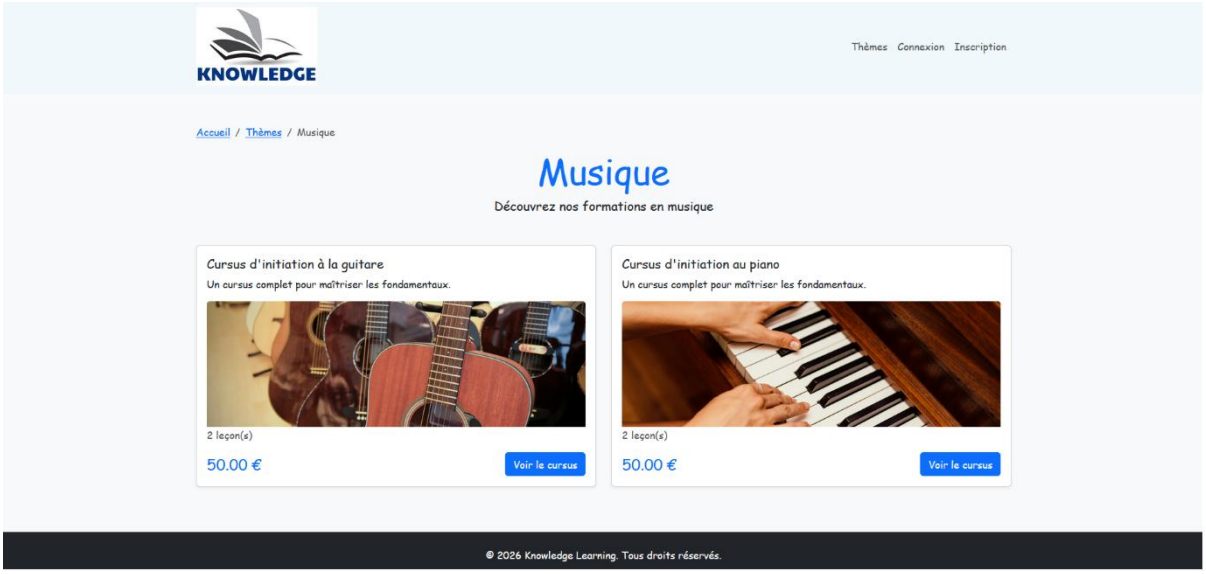


Navigation claire vers les différents thèmes avec cards Bootstrap uniformes.

Mobile –Tablette

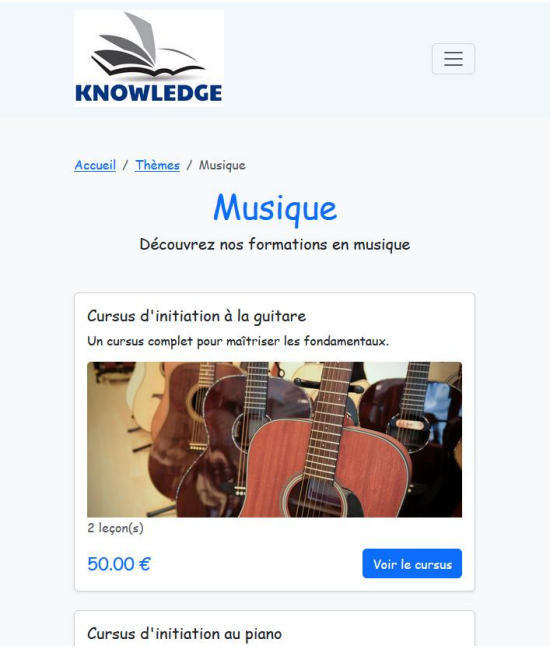


Page détail d'un thème (Musique)

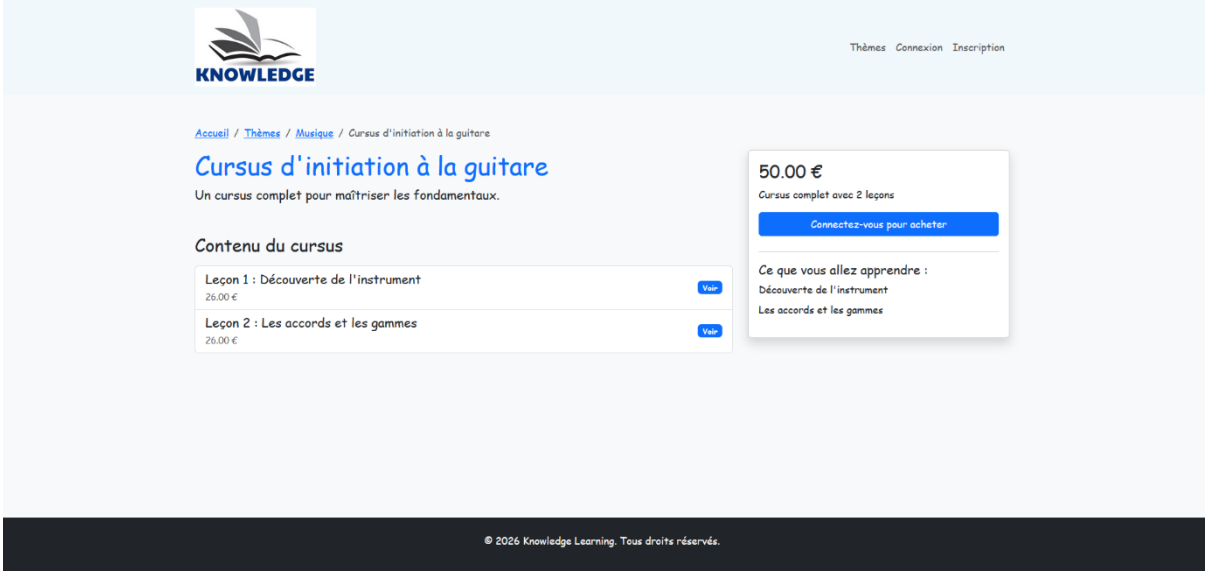


Breadcrumb de navigation + affichage des cours disponibles avec prix et nombre de leçons.

Mobille - tablette

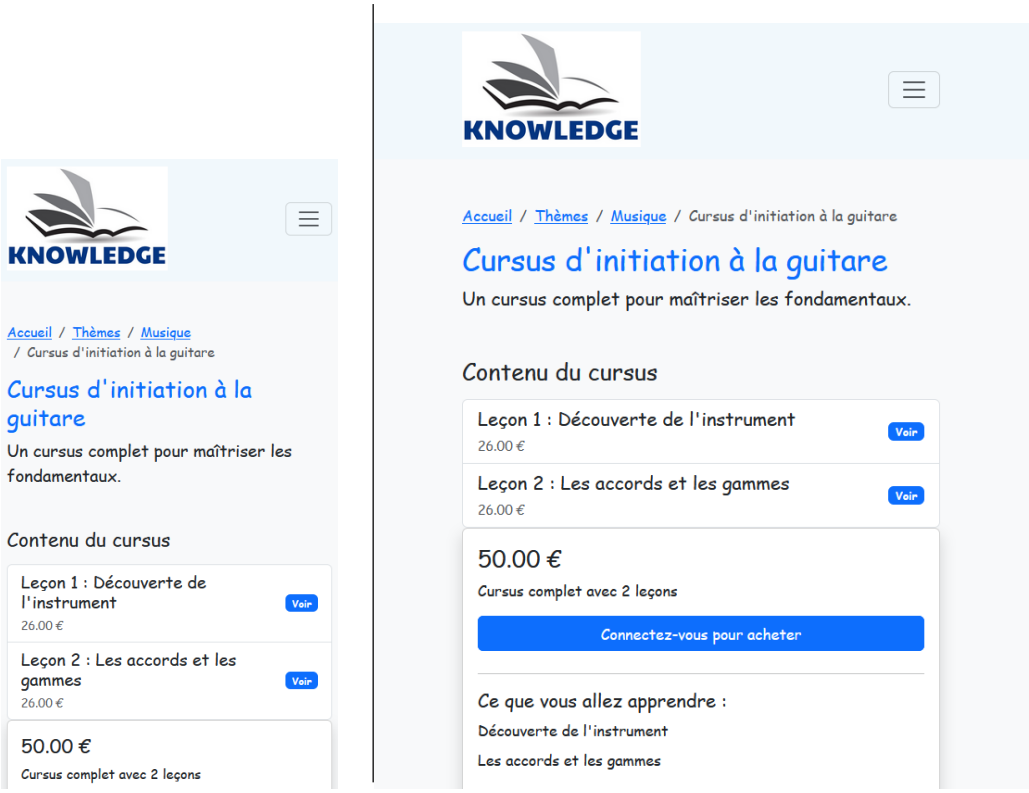


Page détail d'un cursus

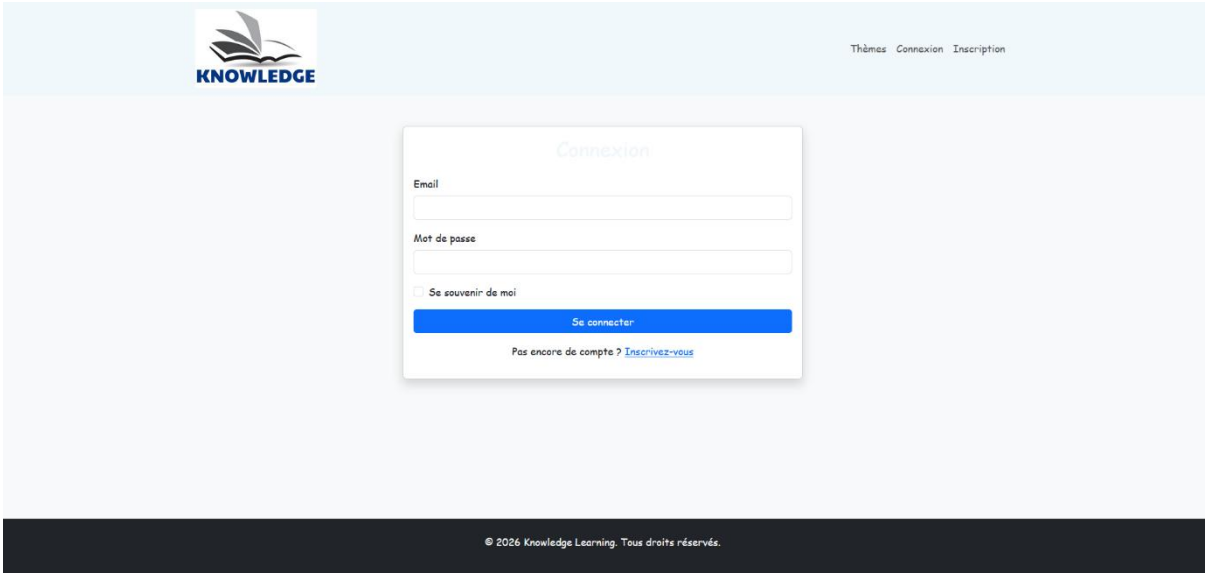


Layout 2 colonnes : contenu principal + sidebar sticky avec prix et CTA d'achat. Logique conditionnelle selon l'état de connexion.

Mobille - tablette

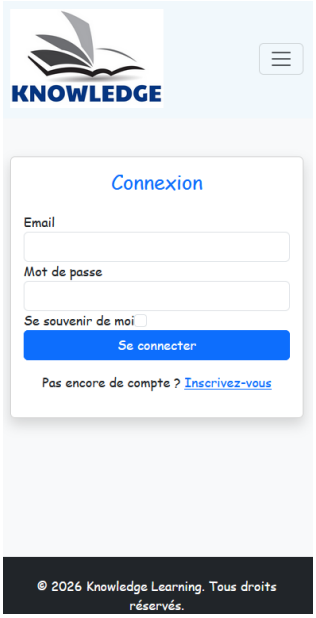


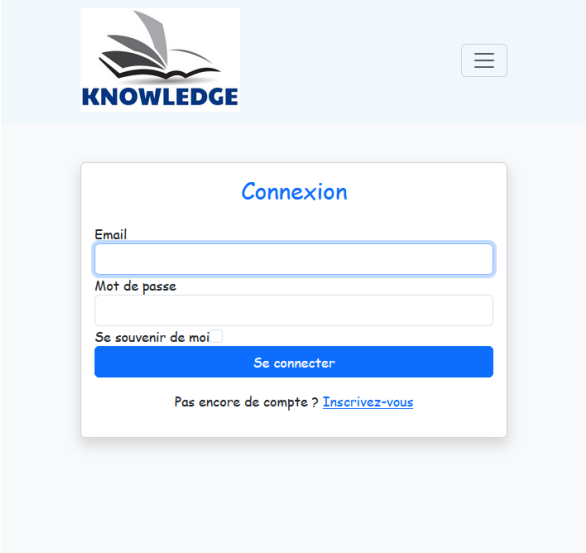
Page de connexion



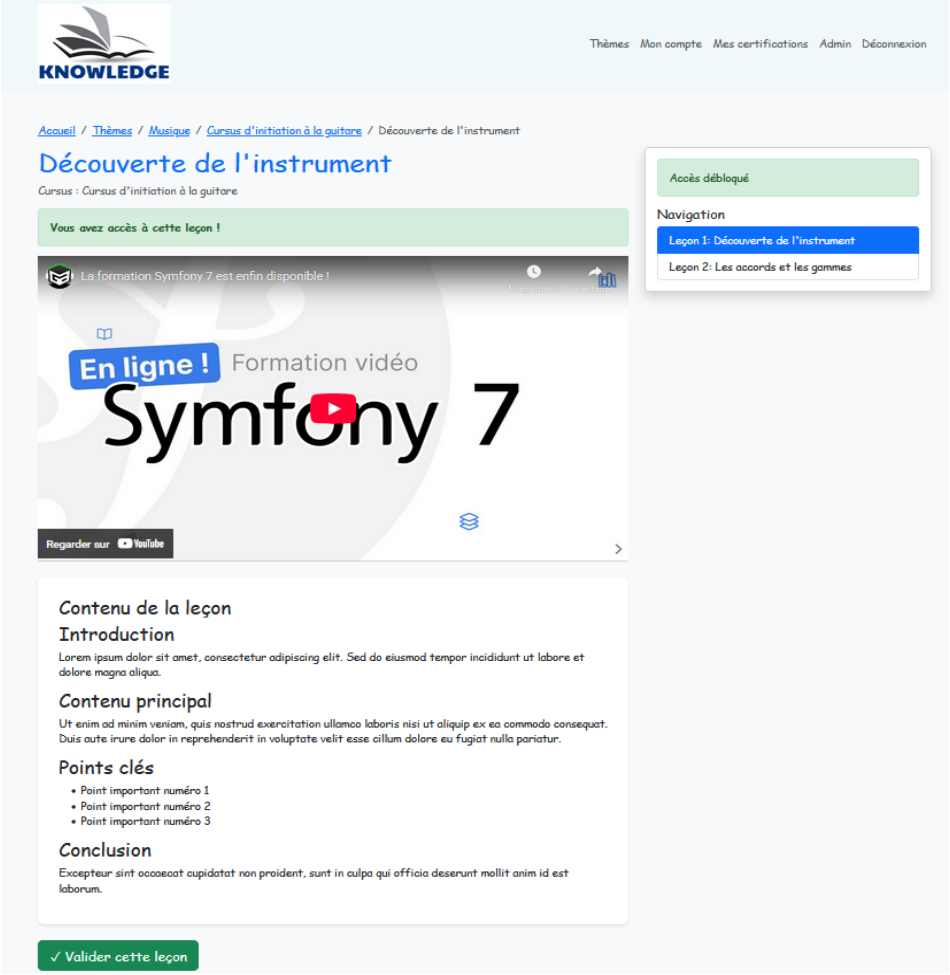
Formulaire Symfony centré avec protection CSRF automatique et fonction "Se souvenir de moi".

Mobille - tablette



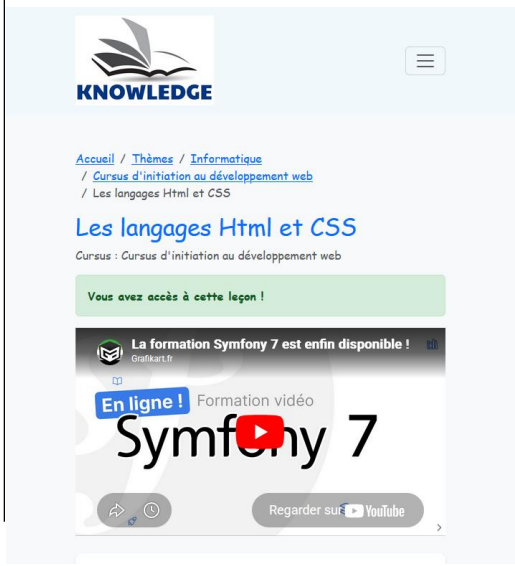


Contenu d'une leçon (accès authentifié)



Vidéo YouTube embarquée + contenu structuré + bouton de validation pour progression. Sidebar de navigation entre leçons.

Mobille - tablette



5.2 Schéma de navigation

Un schéma d'enchaînement des écrans a été élaboré pour visualiser la navigation entre les différentes interfaces :

Parcours utilisateur public :

- authentification → accueil → choix d'un cursus → achat → consultation des leçons → progression → certification.

Côté administrateur :

- connexion → tableau de bord → gestion des utilisateurs/contenus → suivi des commandes.

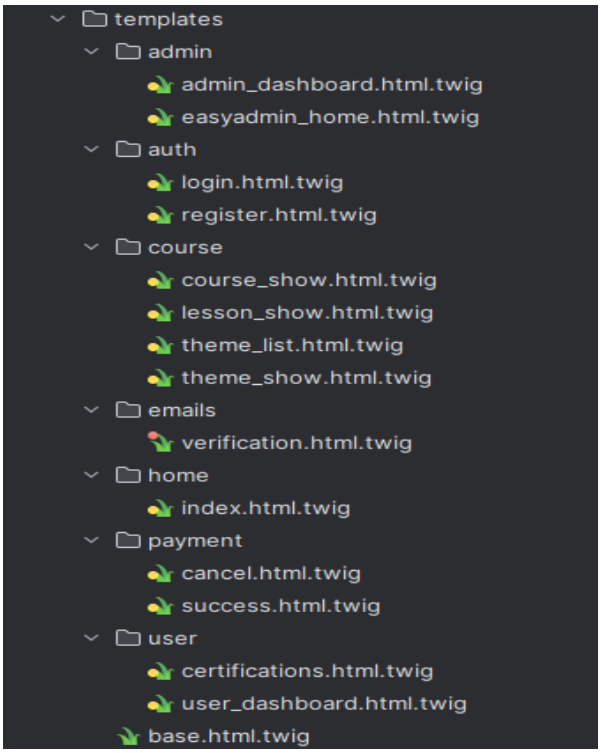
Ce schéma garantit une **logique de parcours claire et cohérente** pour les deux types d'utilisateurs.

5.3 Développement et intégration

Le front-end a été implémenté en **Twig**, en adoptant une **architecture basée sur l'héritage de templates**. Chaque partie de l'interface (formulaire d'inscription, tableau de bord, liste des cours, panier, etc.) a été développée comme un template réutilisable, facilitant ainsi la maintenabilité et l'évolution du projet.

Afin de garantir une **organisation claire du projet** et une **bonne séparation des responsabilités**, la structure des fichiers respecte les bonnes pratiques de Symfony.

L'illustration ci-dessous montre l'arborescence du projet front-end :



5.4 Architecture des templates

Le template de base (base.html.twig) :

Ce template gère l'affichage de la navigation, du contenu principal et du footer.

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Knowledge Learning{% endblock %}</title>
    <link rel="icon" href="{{ asset('favicon.ico') }}">
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
    <link rel="stylesheet" href="{{ asset('styles/app.css') }}">
    {% block stylesheets %}{% endblock %}
            {{ encore_entry_link_tags('app') }}
</head>
<body class="d-flex flex-column min-vh-100">
<nav class="navbar navbar-expand-lg navbar-light bg-primary-custom">
    <div class="container">
        <a class="navbar-brand fw-bold" href="{{ path('/') }}">
            <img src="{{ asset('images/logo.jpg') }}" alt="Knowledge Learning" height="125">
        </a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav ms-auto">
                <li class="nav-item">
                    <a class="nav-link" href="{{ path('/themes') }}">Thèmes</a>
                </li>
                {% if app.user %}
                    <li class="nav-item">
                        <a class="nav-link" href="{{ path('/mon-compte') }}">Mon compte</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="{{ path('/mes-certifications') }}">Mes certifications</a>
                    </li>
                    {% if is_granted('ROLE_ADMIN') %}
                        <li class="nav-item">
                            <a class="nav-link" href="{{ path('/admin') }}">Admin</a>
                        </li>
                    {% endif %}
                    <li class="nav-item">
                        <a class="nav-link" href="{{ path('/logout') }}">Déconnexion</a>
                    </li>
                {% else %}
                    <li class="nav-item">
                        <a class="nav-link" href="{{ path('/connexion') }}">Connexion</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="{{ path('/register') }}">Inscription</a>
                    </li>
                {% endif %}
            </ul>
        </div>
    </div>
</nav>

<main class="container my-5 flex-fill">
    {% for label, messages in app.flashes %}
        {% for message in messages %}
            <div class="alert alert-{{ label }} alert-dismissible fade show" role="alert">
                {{ message }}
                <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
            </div>
        {% endfor %}
    {% endfor %}

    {% block body %}{% endblock %}
</main>
{% block footer %}
<footer class="bg-dark text-white text-center py-4 mt-auto">
    <div class="container">
        <p>© {{ 'now'|date('Y') }} Knowledge Learning. Tous droits réservés.</p>
    </div>
</footer>
{% endblock %}

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
<script src="{{ asset('app.js') }}"></script>
{% block javascripts %}{% endblock %}
            {% block importmap %}{{ importmap('app') }}{% endblock %}
            {{ encore_entry_script_tags('app') }}
</body>
</html>
```

Les templates enfants héritent du layout principal :

```
{% extends 'base' %}

{% block title %}{{ theme.name }} - Knowledge Learning{% endblock %}

{% block body %}
<nav aria-label="breadcrumb">
    <ol class="breadcrumb">
        <li class="breadcrumb-item"><a href="{{ path('/') }}">Accueil</a></li>
        <li class="breadcrumb-item"><a href="{{ path('/themes') }}">Thèmes</a></li>
        <li class="breadcrumb-item active">{{ theme.name }}</li>
    </ol>
</nav>

<div class="text-center mb-5">
    <h1 class="display-4 text-primary">{{ theme.name }}</h1>
    {% if theme.description %}
        <p class="lead">{{ theme.description }}</p>
    {% endif %}
</div>

<div class="row">
    {% for course in courses %}
        <div class="col-md-6 mb-4">
            <div class="card h-100 shadow-sm hover-card">
                <div class="card-body">
                    <h5 class="card-title">{{ course.title }}</h5>
                    <p class="card-text">{{ course.description|default('Découvrez ce cursus complet.') }}</p>
                    <img src="{{ asset('images/courses/' ~ course.slug ~ '.jpg') }}"
                         class="card-img-top img-card-cover"
                         alt="{{ course.title }}">
                    <p class="text-muted">{{ course.lessons|length }} leçon(s)</p>
                    <div class="d-flex justify-content-between align-items-center">
                        <span class="h4 text-primary mb-0">{{ course.price }} €</span>
                        <a href="{{ path('/cursus/{slug}', {slug: course.slug}) }}" class="btn btn-primary">
                            Voir le cursus
                        </a>
                    </div>
                </div>
            </div>
        </div>
    {% else %}
        <div class="col-12">
            <div class="alert alert-info">Aucun cursus disponible pour ce thème.</div>
        </div>
    {% endfor %}
</div>
{% endblock %}
```

5.5 Gestion de la navigation et des données

Les contrôleurs Symfony dans src/Controller/ gèrent la logique métier et passent les données aux templates :

```php
<?php

namespace App\Controller\Course;

use ...

Mathieu P.
#[Route('/theme/{slug}', name: 'app_theme_show')]
class ThemeShowController extends AbstractController
{
    Mathieu P.
    public function __invoke(string $slug, CourseService $courseService): Response
    {
        $theme = $courseService->getThemeBySlug($slug);

        if (!$theme) {
            throw $this->createNotFoundException('Ce thème n\'existe pas.');
        }

        $courses = $courseService->getCoursesByTheme($theme);

        return $this->render( view: 'course/theme_show.html.twig', [
            'theme' => $theme,
            'courses' => $courses,
        ]);
    }
}
```

Adaptation responsive

L'adaptation responsive a été assurée grâce à l'utilisation des propriétés **CSS modernes (Flexbox et Grid Layout)** de Bootstrap 5, qui permettent de garantir un rendu homogène sur différents supports.

Breakpoints :

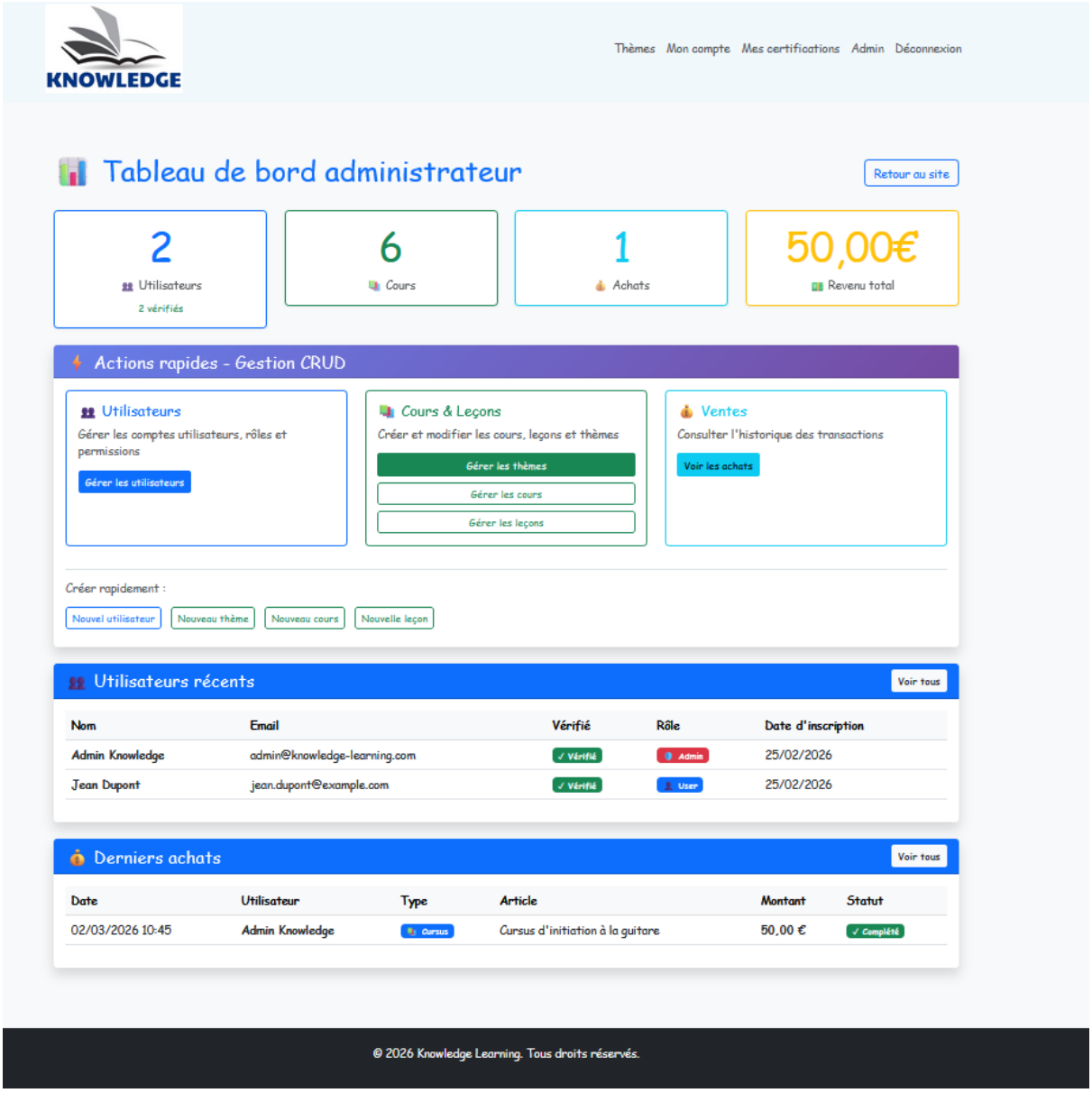
- Mobile (< 768px) : 1 colonne
- Tablette (≥ 768px) : 2 colonnes
- Desktop (≥ 992px) : 4 colonnes

Navigation mobile :

```html
<button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
    <span class="navbar-toggler-icon"></span>
</button>
<div class="collapse navbar-collapse" id="navbarNav">
```

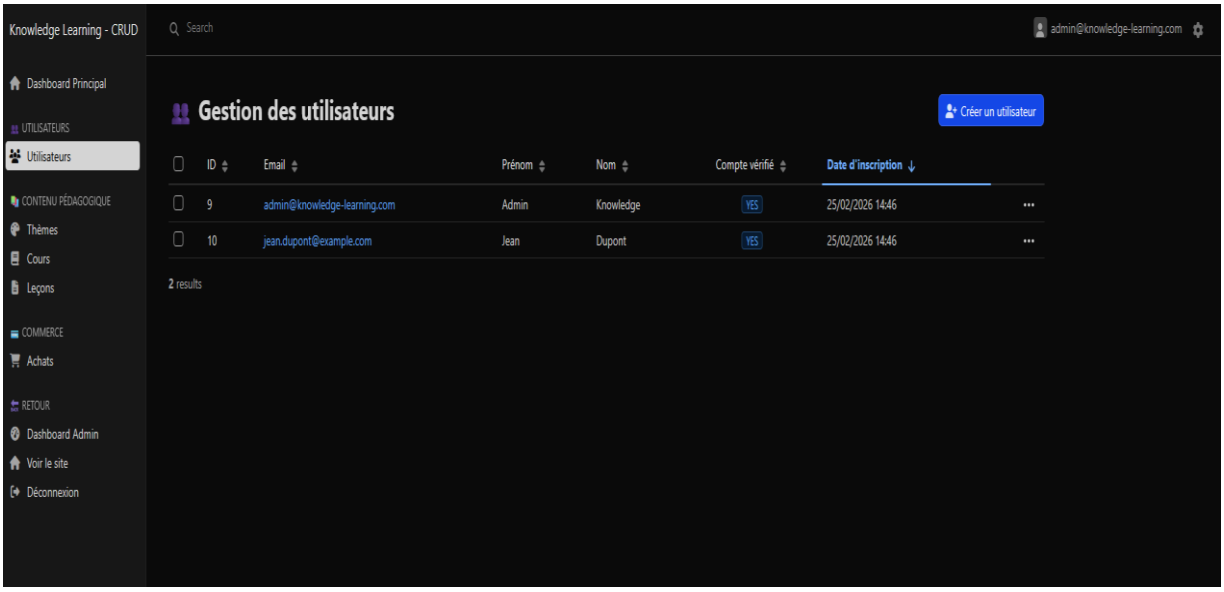
5.6 Pages d'administration

Dashboard administrateur personnalisé



Statistiques en temps réel (utilisateurs, cours, achats, revenus) + actions rapides CRUD + tables des utilisateurs et achats récents.

Interface EasyAdmin pour gestion CRUD



Interface professionnelle avec sidebar de navigation, table triable et recherche en temps réel pour la gestion des utilisateurs, cours, leçons et thèmes.

Accessibilité et inclusion

Une attention particulière a été portée à l'**accessibilité (A11y)** pour garantir que la plateforme Knowledge Learning soit utilisable par tous. La conception des interfaces a tenu compte des ratios de contraste recommandés (WCAG) pour assurer une bonne lisibilité.

Le clavier a été testé sur l'ensemble des parcours, des formulaires à la consultation des leçons.

Exemples d'implémentation :

```
<nav aria-label="breadcrumb">
    <ol class="breadcrumb">
        <li class="breadcrumb-item"><a href="{{ path('/') }}">Accueil</a></li>
        <li class="breadcrumb-item"><a href="{{ path('/themes') }}">Thèmes</a></li>
        <li class="breadcrumb-item"><a href="{{ path('/theme/{slug}', {slug: lesson.course.theme.slug}) }}">{{ lesson.course.theme.name }}</a></li>
        <li class="breadcrumb-item"><a href="{{ path('/cursus/{slug}', {slug: lesson. course.slug}) }}">{{ lesson.course.title }}</a></li>
        <li class="breadcrumb-item active">{{ lesson.title }}</li>
    </ol>
</nav>
```

Gestion des assets (Webpack Encore)

La gestion des assets CSS et JavaScript est assurée par **Webpack Encore**, permettant :

- ✅ Minification et optimisation en production
- ✅ Hot reload en développement
- ✅ Versioning automatique (cache busting)
- ✅ Code splitting (vendor séparé)

```
Encore
    .setOutputPath('public/build/')
    .setPublicPath('/build')
    .addEntry( name: 'app',  src: './assets/app.js')
    .enableStimulusBridge( controllerJsonPath: './assets/controllers.json')
    .splitEntryChunks()
    .enableVersioning(Encore.isProduction());
```

6. Synthèse des compétences professionnelles

N° CP	Compétence professionnelle	Réalisation dans le projet	Page
1	Installer et configurer son environnement	XAMPP, Composer, Git, Symfony CLI, Webpack Encore	p.8
2	Maquetter des interfaces	Bootstrap 5, 8 interfaces responsive, charte graphique	p.15-16
3	Réaliser interfaces statiques	HTML5 sémantique, 15 templates Twig, Bootstrap components	p.17-19
4	Développer interfaces dynamiques	Twig (conditions, boucles), Symfony Forms, Stimulus	p.19-20
5	Mettre en place BDD relationnelle	MySQL, 7 entités Doctrine, relations, migrations	p.10-12
6	Composants d'accès aux données	Doctrine ORM, Repositories, QueryBuilder, Fixtures	p.12-13
7	Composants métier serveur	15 contrôleurs, Services (Payment, Progression), Authentification	p.13-14
8	Documenter le déploiement	README, scripts SQL, .env.example, documentation sécurité	p.9, Annexes

7. ANNEXES

5.1 Instructions d'Installation

Voir **README.md** du repository pour :

- Prérequis (PHP 8.2, Composer, MySQL, Node.js)
- Installation des dépendances
- Configuration de l'environnement (.env.local)
- Création de la base de données
- Chargement des fixtures
- Configuration Stripe
- Lancement de l'application

5.2 Comptes de Test

Créés automatiquement par les fixtures (doctrine:fixtures:load) :

Rôle	Email	Mot de passe
Administrateur	admin@knowledge-learning.com	Admin123!
Utilisateur	jean.dupont@example.com	User123!

5.3 Stack Technique

Backend

- **PHP 8.2+**
- **Symfony 7.4**
- **Doctrine ORM** (gestion BDD)
- **MySQL 8.0** / MariaDB
- **PHPUnit 9** (tests)

Frontend

- **Twig** (templates)
- **Bootstrap 5.3**
- **Stimulus** (interactions JavaScript)
- **Webpack Encore** (bundler)

E-Commerce

- **Stripe API** (paiements)
- **Stripe CLI** (webhooks locaux)

Outils

- **Composer** (dépendances PHP)
- **Yarn** (dépendances JS)
- **Mailpit** (emails de développement)

5.4 Périmètre Fonctionnel Réalisé

✅ Inscription avec activation email
✅ Connexion/Déconnexion
✅ Achat de cursus complets
✅ Achat de leçons individuelles
✅ Gestion des droits d'accès aux contenus
✅ Validation progressive des leçons
✅ Attribution automatique de certifications
✅ Backoffice administrateur (EasyAdmin)
✅ Dashboard utilisateur
✅ Gestion des achats et certifications

5.5 Points d'Amélioration Futurs

- Système de notation/commentaires sur les cours
- Lecteur vidéo intégré (Vimeo/YouTube)
- Export PDF des certifications
- Statistiques de progression avancées
- Mode "prévisualisation" des leçons gratuites
- Système de panier multi-produit

📧 Contact : mathieu.paquier85@gmail.com